

Optimization

3.2.2 Grid search

José Firmino Aguilar Madeira

Random or systematic?

In 3.2.1 we scattered points **at random**.

What if we want to cover the domain **systematically**?

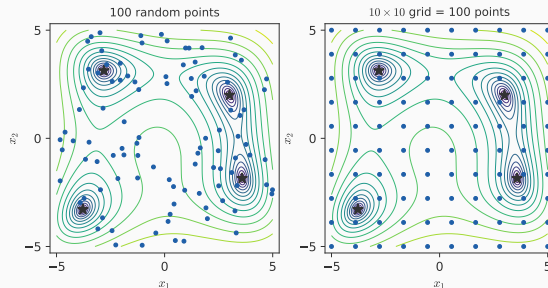
Grid search: m values per variable,

$$x_i^{(1)}, \dots, x_i^{(m)} \text{ in } [a_i, b_i],$$

and evaluate f at **all combinations**:

m^n evaluations.

Himmelblau on $[-5, 5]^2$, the same budget:
100 points.



Random or systematic?

In 3.2.1 we scattered points **at random**.

What if we want to cover the domain **systematically**?

Grid search: m values per variable,

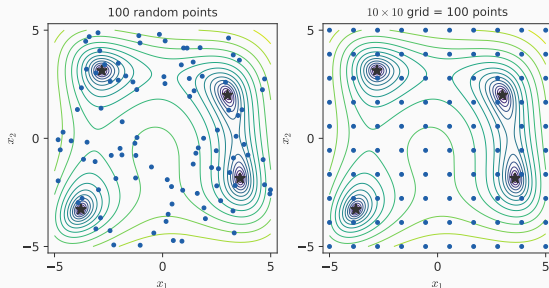
$$x_i^{(1)}, \dots, x_i^{(m)} \text{ in } [a_i, b_i],$$

and evaluate f at **all combinations**:

m^n evaluations.

Himmelblau on $[-5, 5]^2$, the same budget:
100 points.

Which do we prefer?



Random or systematic?

In 3.2.1 we scattered points **at random**.

What if we want to cover the domain **systematically**?

Grid search: m values per variable,

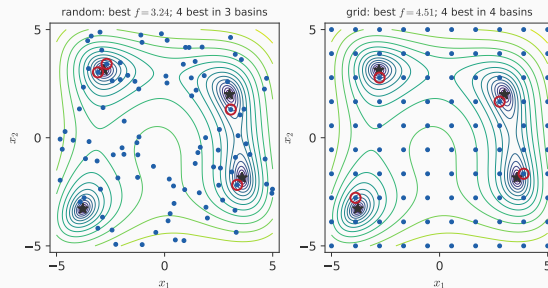
$$x_i^{(1)}, \dots, x_i^{(m)} \text{ in } [a_i, b_i],$$

and evaluate f at **all combinations**:

m^n evaluations.

Himmelblau on $[-5, 5]^2$, the same budget:
100 points.

Which do we prefer?



Grid: reproducible, regular coverage; the 4 best nodes are in the **4 basins**. **Random:** best f in 30 runs, median $[Q1, Q3]$: 2.48 $[1.43, 3.47]$ (grid: 4.51), but the 4 best points lie in 2–3 basins (never in all 4). **There is no universal answer** — on this problem, with this budget.

The price of the grid

m^n evaluations

$m = 10$	$n = 2$	$n = 5$	$n = 10$
evaluations	100	10^5	10^{10}

The curse of dimensionality, again: with $N = m^n$ points, the spacing is

$$h = \frac{b-a}{m-1} \sim N^{-1/n},$$

the same law as random search (3.2.1).

- **Refining is not gradual:** going from $m = 10$ to $m = 11$ in 10D costs $1.6 \cdot 10^{10}$ more evaluations. In random search points are added **one at a time**.
- **Guarantee:** with f continuous on a bounded domain, the best grid value approaches the global minimum value, $f_{\text{grid}} \rightarrow f^*$, as the grid is refined. If f is L -Lipschitz (R) on a rectangular domain covered by the grid, $f_{\text{grid}} - f^* \leq L \frac{\sqrt[n]{n}}{2} h$.
- Box (3.2.4) can be seen as a *local* grid with $m = 2$ per variable, around the current point: 2^n vertices.

What is it really useful for?

Not for finding the minimum accurately, but for

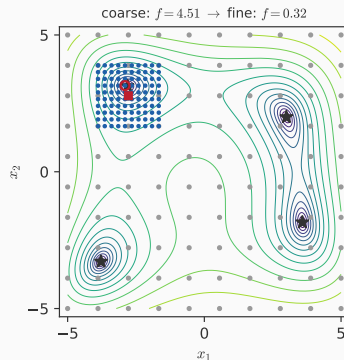
exploring → identifying promising regions → initializing

- **Visualizing:** the level curves in these lectures come from f on a grid.
- **Coarse** → **fine:** coarse grid, best region, finer grid only there. On Himmelblau: $f = 4.51$ on the 10×10 ; 100 more evaluations around the best node give $f = 0.32$.
- **Initializing** a local method (next page).

In MATLAB:

```
[X1, X2] = meshgrid(linspace(a1,b1,m), ...  
                   linspace(a2,b2,m));  
F = arrayfun(@(u,v) f([u v]), X1, X2);  
[fmin, i] = min(F(:)); x = [X1(i) X2(i)];
```

`meshgrid` is the function that builds the grid; the method is *grid search*.

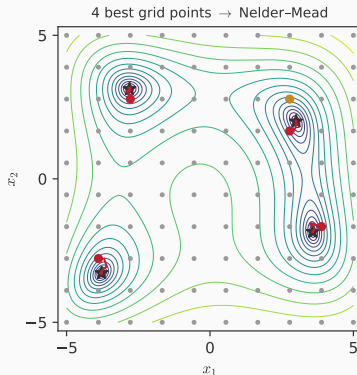


Grey: the 10×10 grid; blue: the fine grid around the best node.

Coarse global $\rightarrow$ fine local

1. 10×10 grid on Himmelblau: 100 evaluations.
2. Select the 4 best nodes: $f = 4.51, 4.70, 6.15, 11.88$ — one in each basin.
3. Nelder–Mead (3.2.3) from each one: $n_f = 89, 94, 94, 87$.

Result: the 4 global minima, all with $f < 10^{-11}$, in 464 evaluations in total.



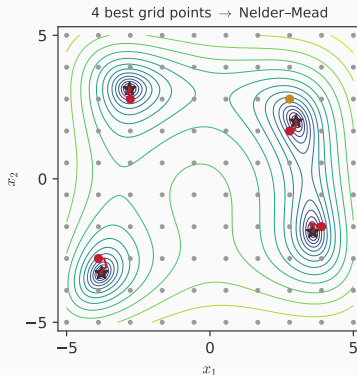
Red: paths of the best Nelder–Mead vertex from the 4 best nodes (the 5th, in orange).

Coarse global $\rightarrow$ fine local

1. 10×10 grid on Himmelblau: **100** evaluations.
2. Select the **4 best** nodes: $f = 4.51, 4.70, 6.15, 11.88$ — one in each basin.
3. Nelder–Mead (3.2.3) from each one: $n_f = 89, 94, 94, 87$.

Result: the **4 global minima**, all with $f < 10^{-11}$, in 464 evaluations in total.

But the 5th best node leads again to (3,2): 95 more evaluations for a minimum already found. Choosing good, *distinct* starting points is a problem in itself — it returns in GLODS (5.1).



Red: paths of the best Nelder–Mead vertex from the 4 best nodes (the 5th, in orange).

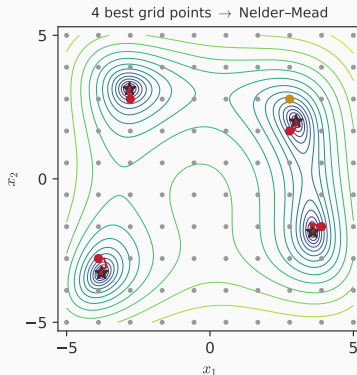
Coarse global $\rightarrow$ fine local

1. 10×10 grid on Himmelblau: 100 evaluations.
2. Select the 4 best nodes: $f = 4.51, 4.70, 6.15, 11.88$ — one in each basin.
3. Nelder–Mead (3.2.3) from each one: $n_f = 89, 94, 94, 87$.

Result: the 4 global minima, all with $f < 10^{-11}$, in 464 evaluations in total.

But the 5th best node leads again to (3,2): 95 more evaluations for a minimum already found. Choosing good, *distinct* starting points is a problem in itself — it returns in GLODS (5.1).

coarse global exploration + local refinement



Red: paths of the best Nelder–Mead vertex from the 4 best nodes (the 5th, in orange).

KEY TAKEAWAYS — 3.2.2

1. Grid search: m values per variable, **all combinations**: m^n evaluations; deterministic and reproducible.
2. It covers the domain **regularly**, but suffers from the dimension like random search (spacing $\sim N^{-1/n}$), and refining is not gradual.
3. With f continuous, the best grid value approaches the global minimum value, $f_{\text{grid}} \rightarrow f^*$, as the grid is refined — at the cost of m^n .
4. It is not a competitor of local methods: it serves to **visualize, explore and initialize**.
5. **Coarse global + fine local**: grid $\rightarrow$ best points $\rightarrow$ local method — an idea that returns in 5.1.

Next (3.2.3): the grid finds promising regions, but keeps spending evaluations where we already know it does not matter. Can we make the **evaluated points themselves move** to where f seems to improve? — the Nelder–Mead simplex.

References and resources

References Full list: *Course bibliography*, at the end of the course PDF.

- Conn, A. R., Scheinberg, K., Vicente, L. N., *Introduction to Derivative-Free Optimization*, SIAM (MPS-SIAM Series on Optimization 8), 2009 — Ch. 1.
- Deb, K., *Optimization for Engineering Design: Algorithms and Examples*, 2nd ed., PHI Learning, 2012 — Ch. 2, exhaustive search.

Resources

- MATLAB: `meshgrid`, `arrayfun`, `min` (grid in the deck itself; no course function).